
Automates finis

Automates finis non déterministe

Propriétés des langages acceptés par un automate fini

Automate déterministe minimal

Automates finis et langages réguliers

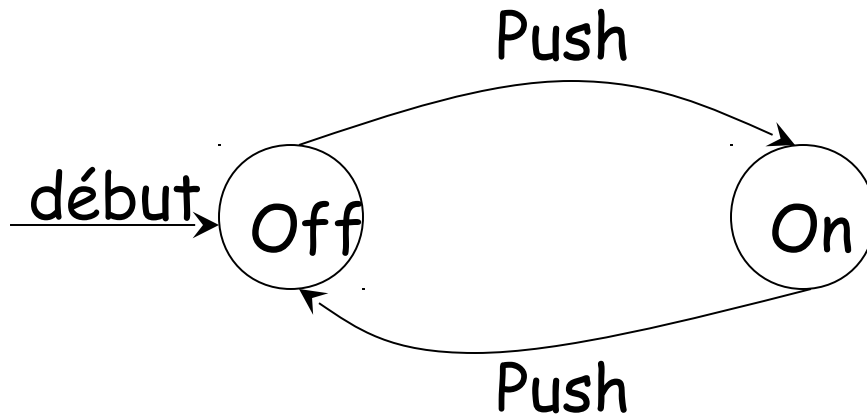
Transformation d'un DFA en ER

Lemme de pompage

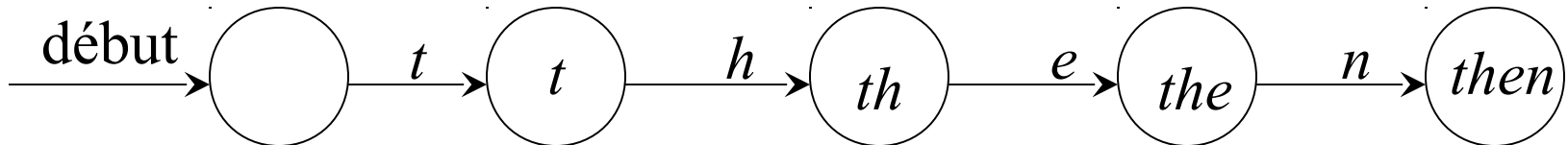
Les automates à états finis sont utilisés comme modèle pour :

- Conception des circuits logiques
- Analyse lexicale de compilateurs
- La recherche de mots clés dans le Web.
-

- Automate fini modélisant un système de On/Off



- Automate fini reconnaissant la chaîne *then*



Problème: est ce qu'une chaîne w appartient à un langage L ?

Quels sont les ressources nécessaires pour répondre à cette question.

Reconnaisseur d'un langage

Programme qui prend en entrée une chaîne x et répond par **oui** ou **non**. (x appartient ou non au langage)

On peut raisonner pour les problèmes non pas comme une réponse Oui /non mais comme transformation des entrées en des sorties.

Automate à états finis déterministe

définition formelle

- Ensemble fini d'états Q
- Alphabet de symboles d'entrées Σ
- Un état initial (un élément de Q) q_0
- Zéro ou plus d'états d'acceptation ou finaux (sous ensemble de Q) F
- Une fonction de transition δ

Une fonction de transition δ

- Arguments : état et un symbole d'entrée
- Retourne un état

$$\delta : Q \times \Sigma \rightarrow Q$$

$$\delta(q_i, a) = q_j$$

- Intuitivement; si un automate fini AF est dans un état q_i et reçoit l'entrée a (reconnaît), alors l'AF passe à l'état q_j

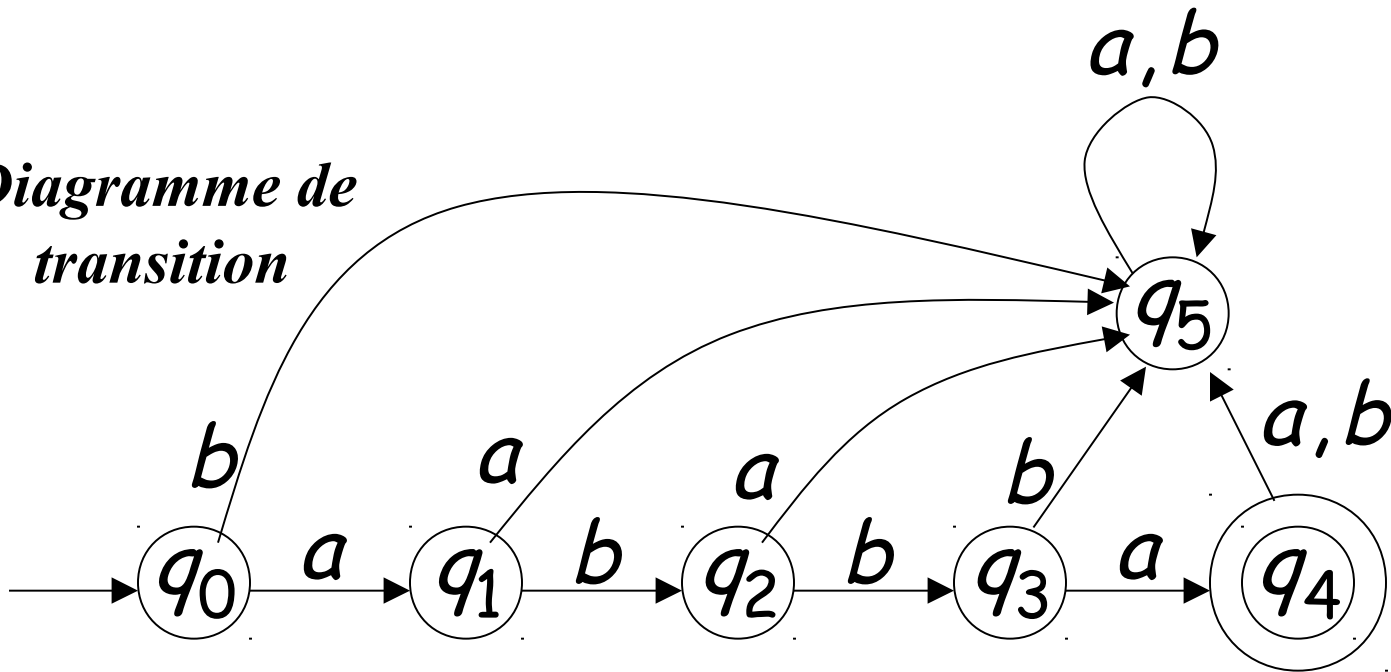
Un Automate à états Finis Déterministe (DFA)

$$M = (Q, \Sigma, \delta, q_0, F)$$

Σ

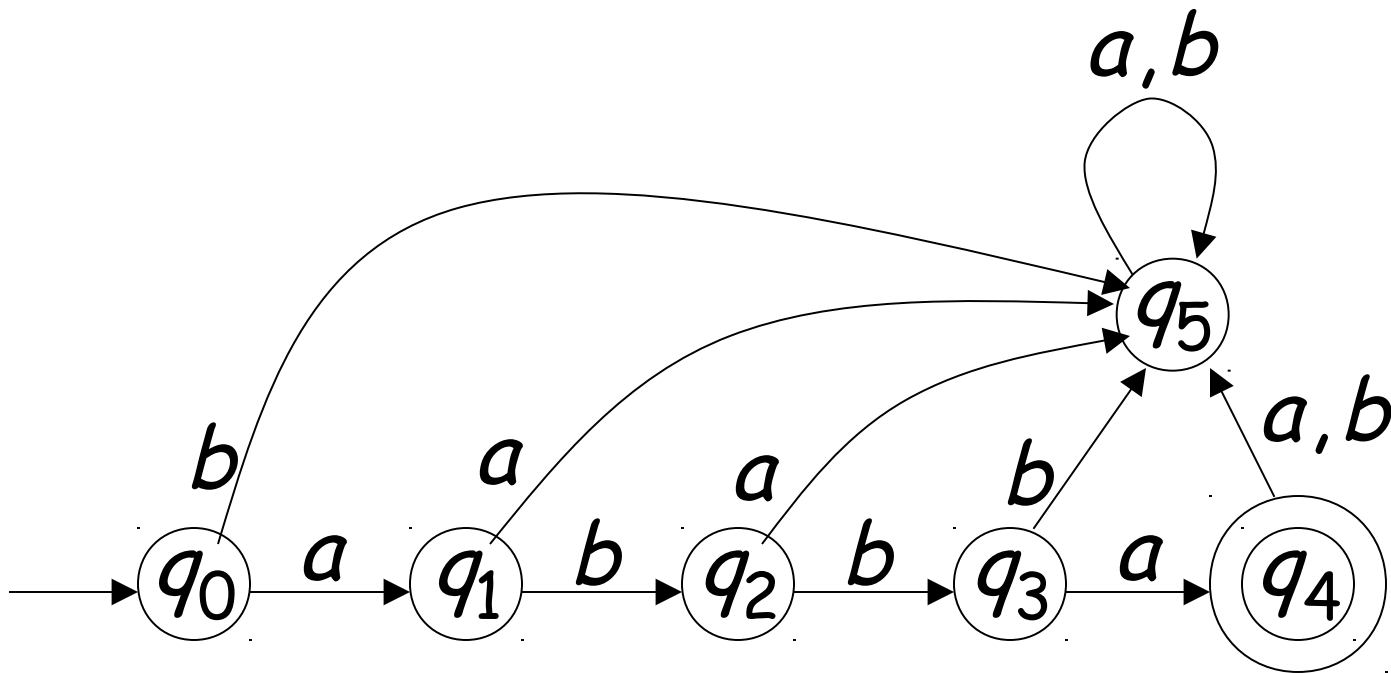
$$\Sigma = \{ a, b \}$$

*Diagramme de
transition*

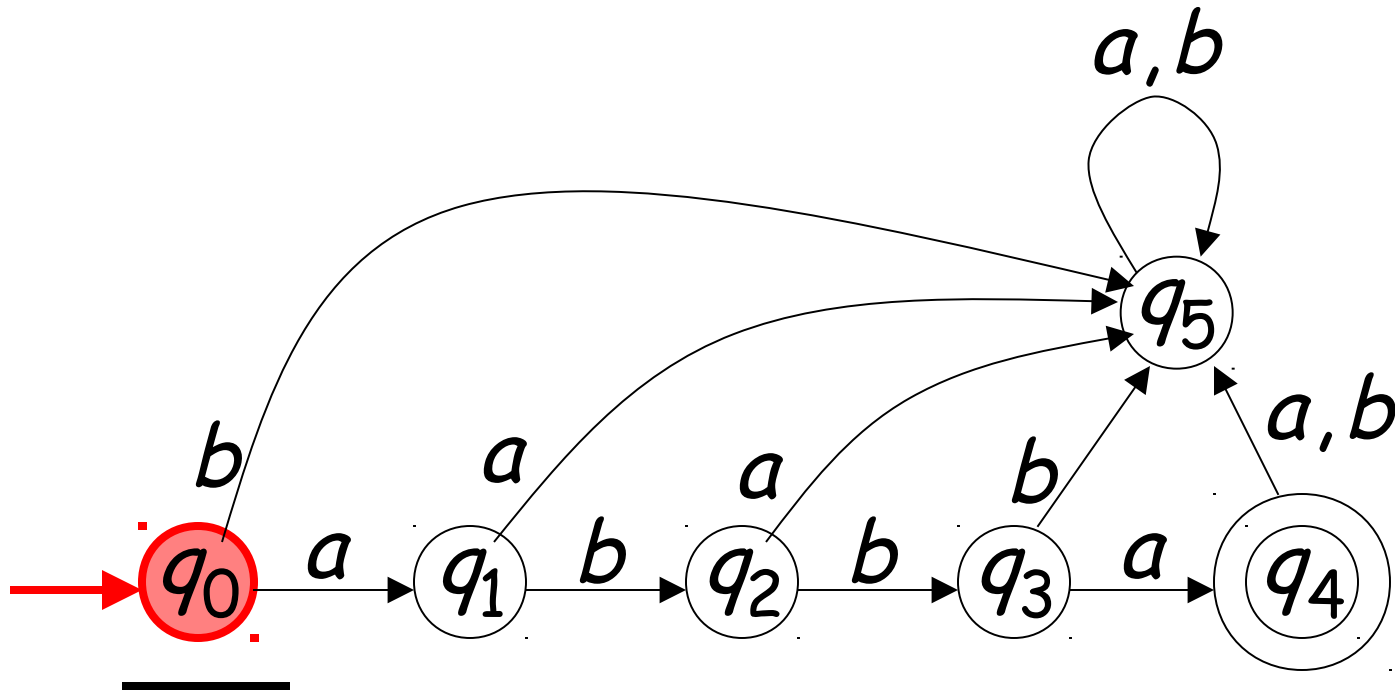


Q

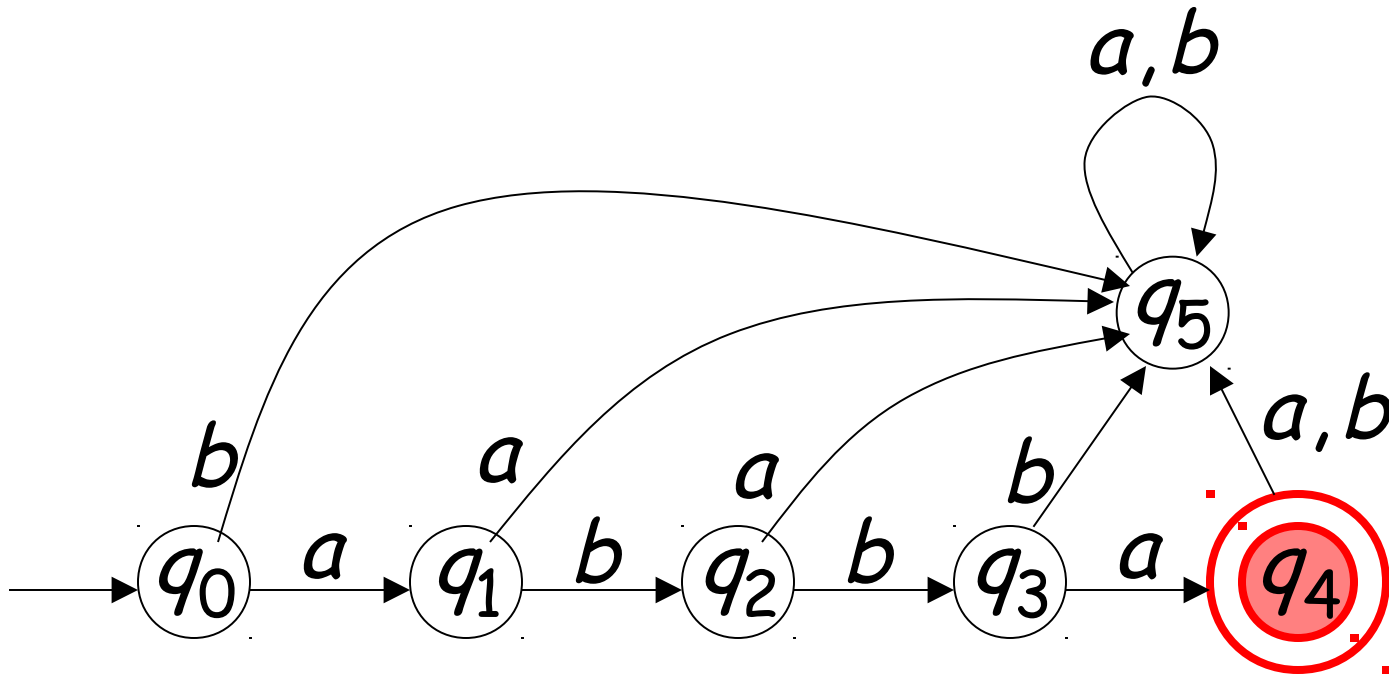
$$Q = \{q_0, q_1, q_2, q_3, q_4, q_5\}$$



q_0



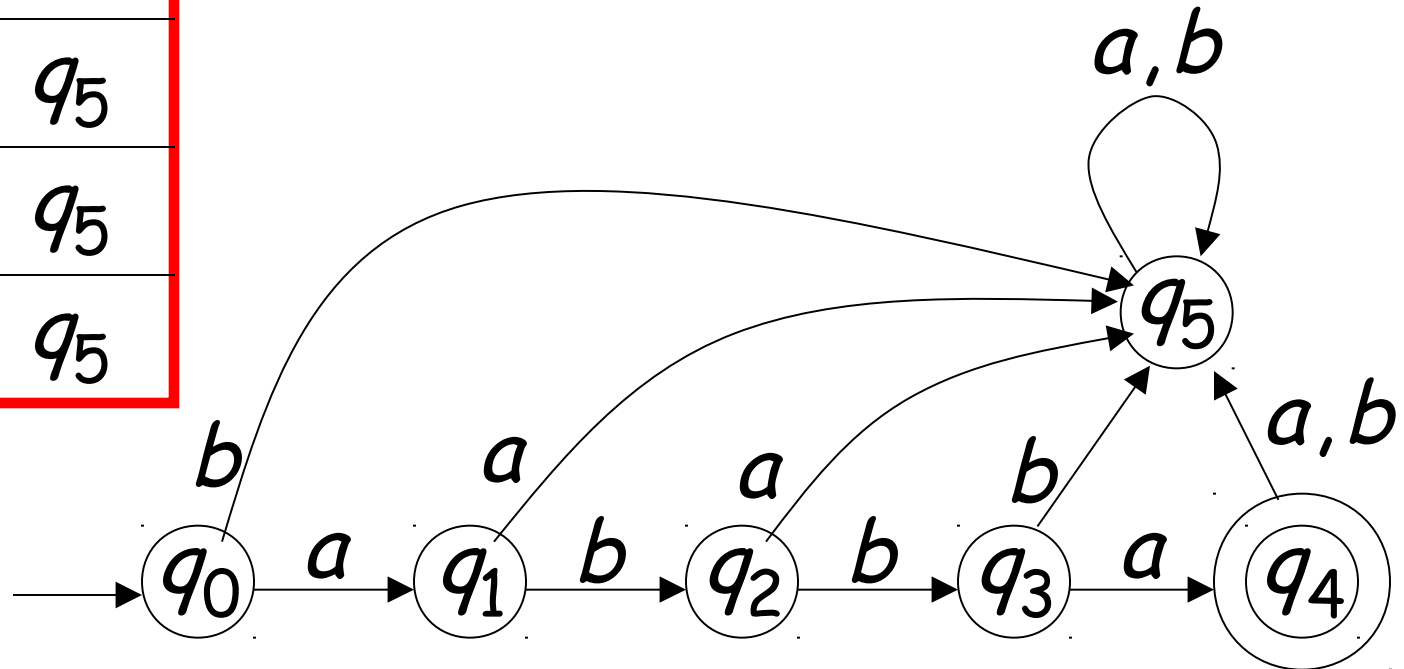
$$F = \{q_4\}$$



δ

δ	a	b
q_0	q_1	q_5
q_1	q_5	q_2
q_2	q_5	q_3
q_3	q_4	q_5
q_4	q_5	q_5
q_5	q_5	q_5

*Mécanisme de calcul
sans mémoire*

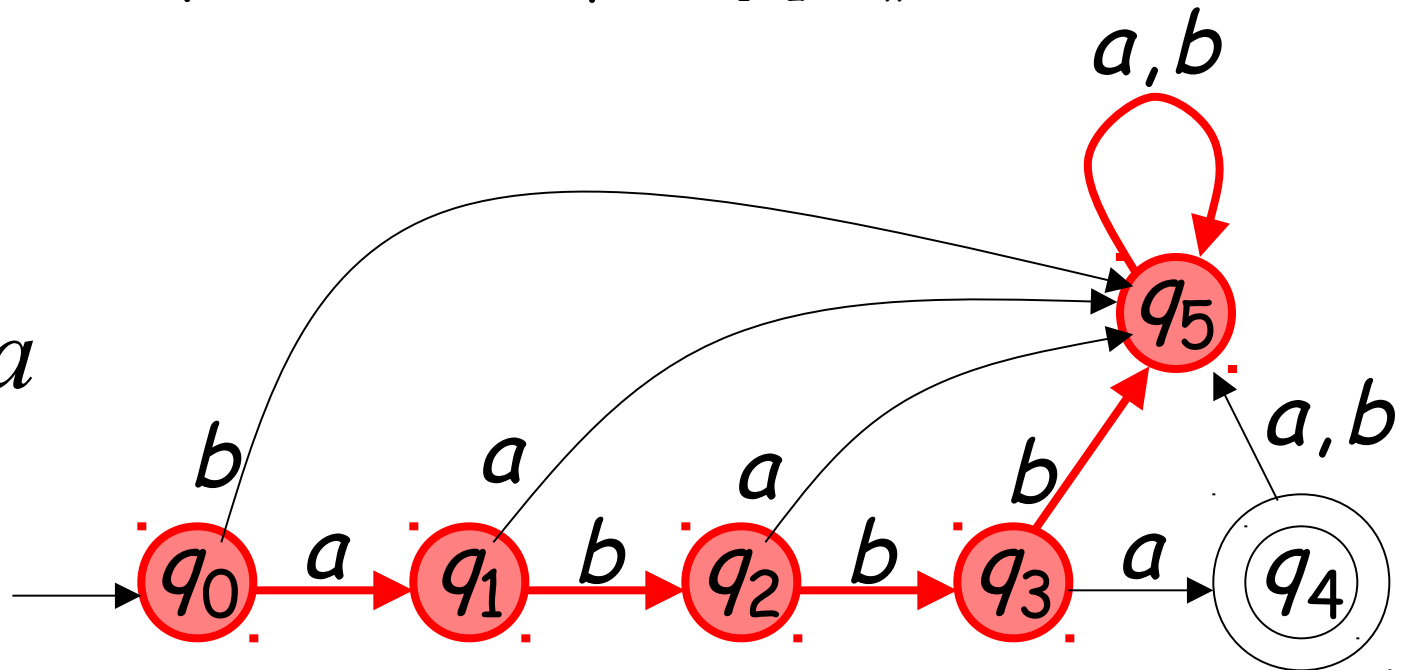


Un automate Fini accepte une chaîne $w = a_1a_2...a_n$ S'il existe un chemin dans le diagramme de transition qui :

1. Commence à l'état initial.
2. Finie à un états d'acceptation (ou final)
3. Possède la séquence libellé par $a_1a_2...a_n$

Exemple

abbbaa



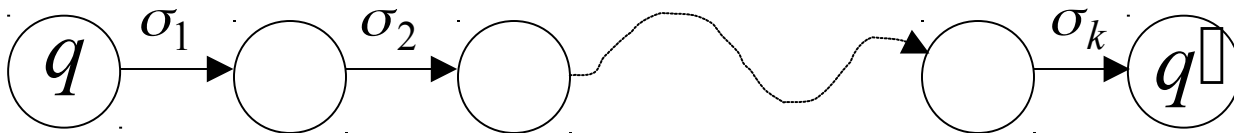
$$\delta^*(q, w) = q'$$

Elle nous dit dans quel état on arrive en partant de l'état q_i
et en analysant la chaîne w

$$\delta^* : Q \times \Sigma^* \rightarrow Q$$

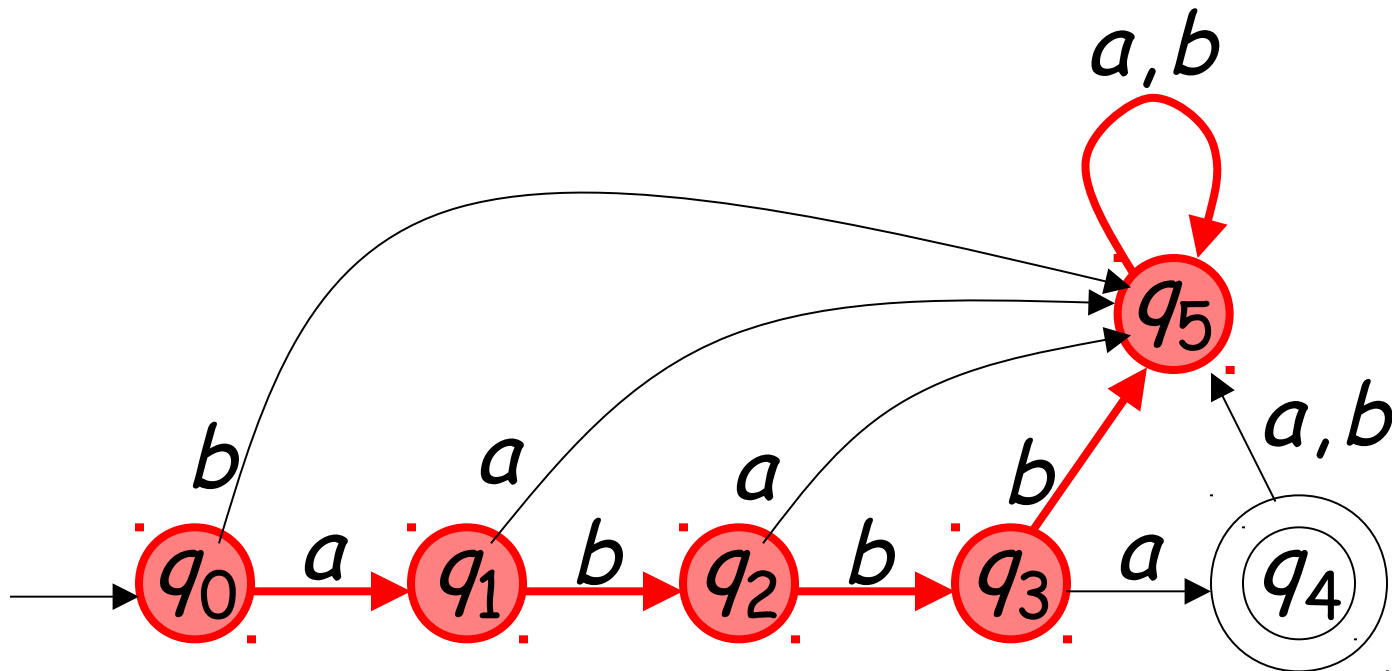


$$w = \sigma_1 \sigma_2 \cdots \sigma_k$$



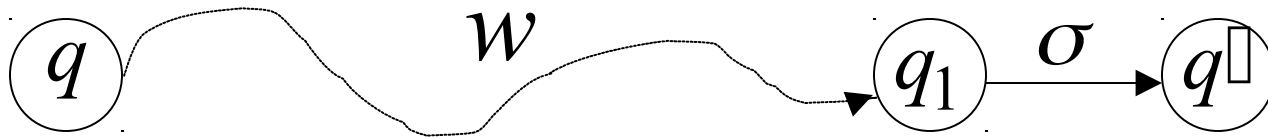
Exemple : Il y a un chemin de q_0 à q_5
libellé *abbbaa*

$$\delta^*(q_0, abbbaa) = q_5$$



Base : $\delta^*(q, \varepsilon) = q$

Induction : $\delta^*(q, w\sigma) = \delta(\delta^*(q, w), \sigma)$



Une chaîne w est **accepté** par l'automate M dont l'état initial est q_0 si $\delta^*(q_0, w)$ appartient aux états finaux de M

Considérons DFA M

Définition:

Le langage $L(M)$ contient toutes les chaînes acceptés par M

$L(M) = \{ \text{toutes les chaînes menant l'automate } M \text{ vers un état final} \}$

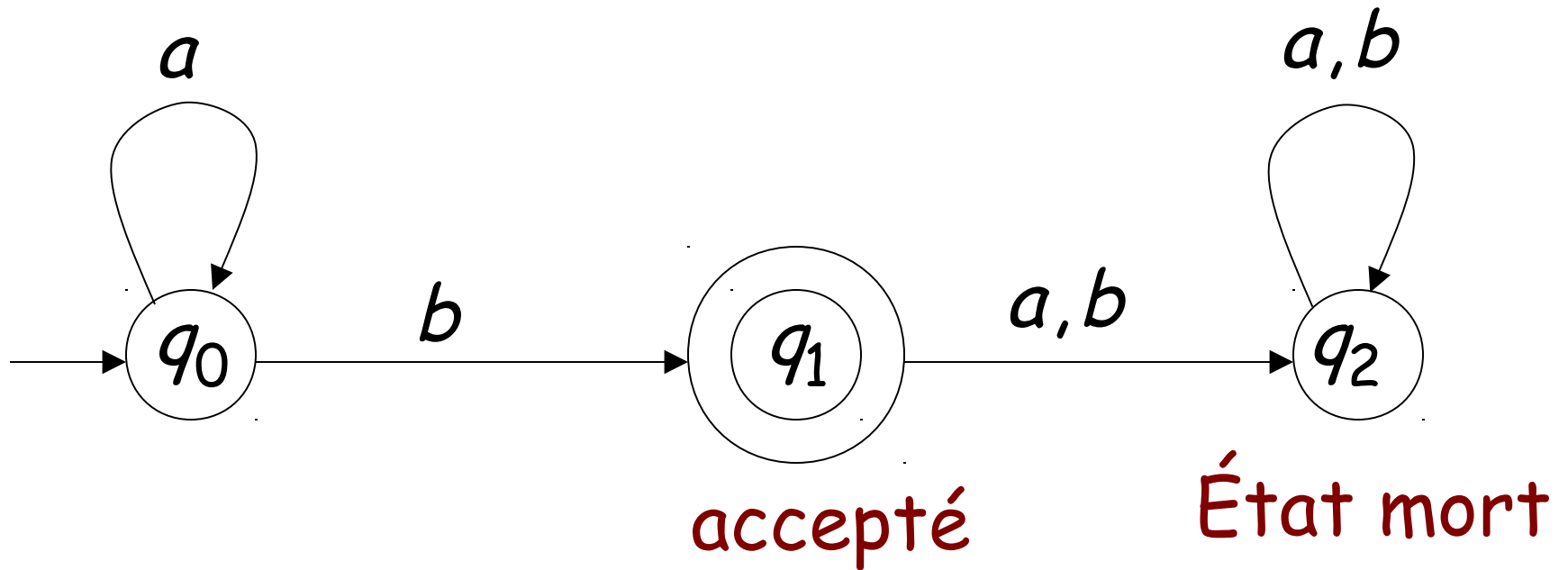
$$M = (Q, \Sigma, \delta, q_0, F)$$

$$L(M) = \{ w \in \Sigma^* : \delta^*(q_0, w) \in F \}$$

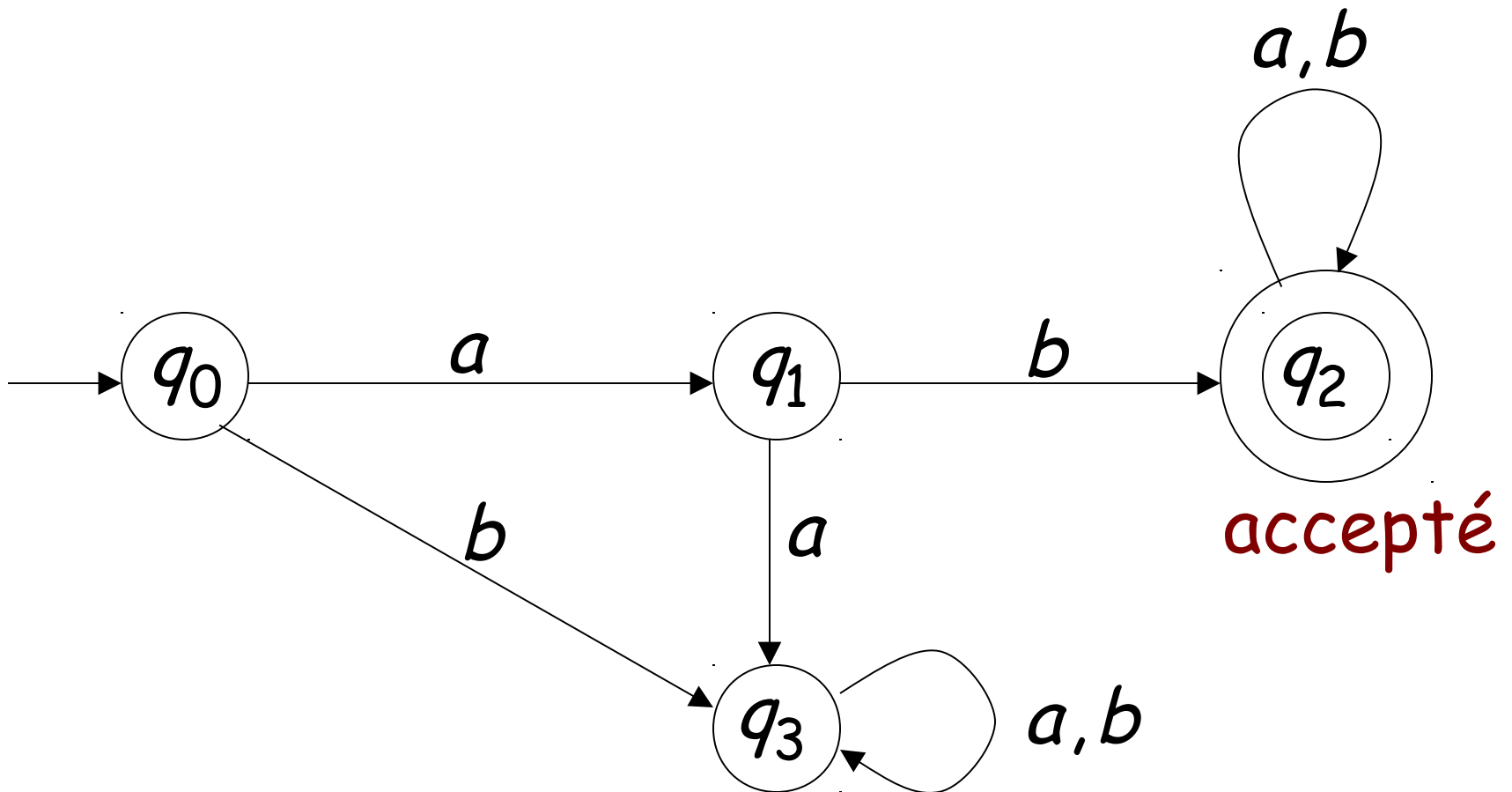
$$\overline{L(M)} = \{ w \in \Sigma^* : \delta^*(q_0, w) \notin F \}$$



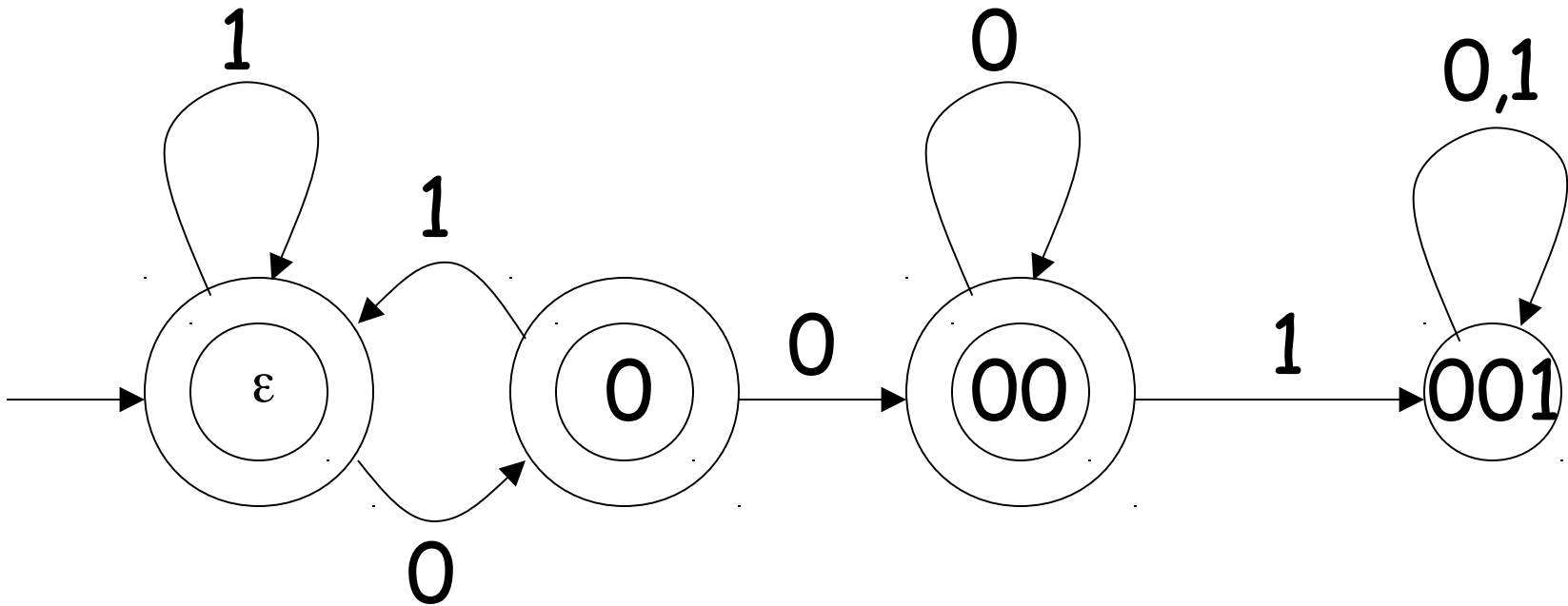
$$L(M) = \{a^n b : n \geq 0\}$$



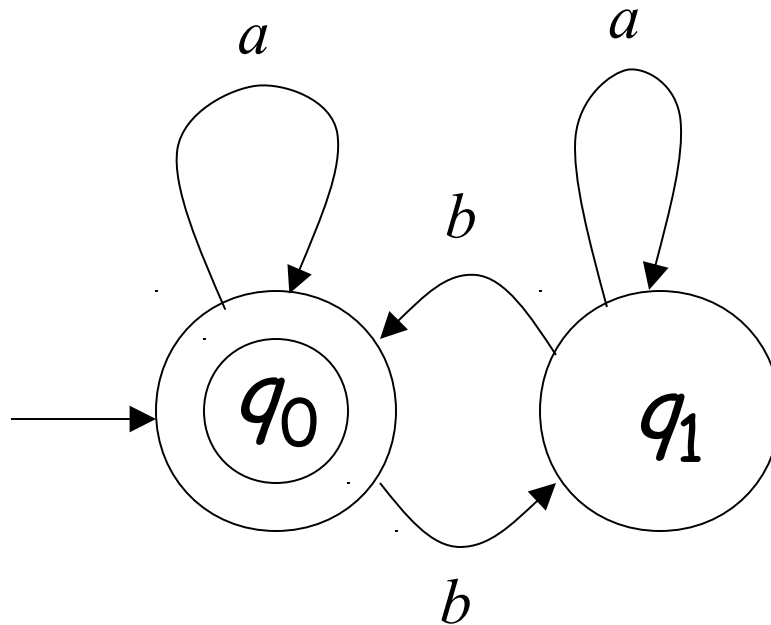
$L(M) = \{ \text{toutes les chaînes avec préfixe } ab \}$



$L(M) = \{ \text{toutes les chaînes sauf celles qui contiennent par } 001 \}$

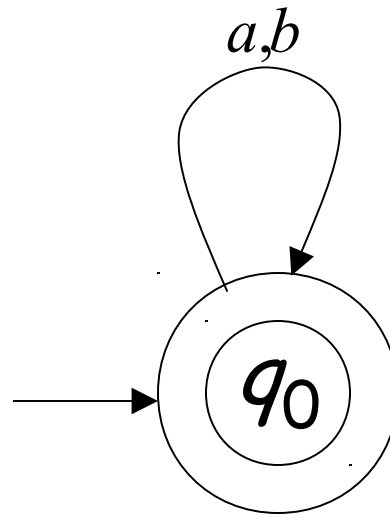


$L(\mathcal{M}) = \{w \text{ contient un nombre pair de } b\}$



q_0 : Est à la fois état initial et final

- Tout les mots sur l'alphabet $\{a,b\}$

 Σ^* 

$L = \{w \in \{a,b\}^* \mid w \text{ contient un nombre pair de } a \text{ et un nombre pair de } b\}$

$L = \{w \in \{0,1,2,3,4,5,6,7,8,9\}^* \mid \text{somme des chiffres est divisible par } 3\}$

$L = \{w \in \{a,b\}^* \mid |w|_b = 3k+1; k \geq 0\}$

Configuration initiale : $\gamma_0(q_0, w)$

Configuration finale d'acceptation : $\gamma_f(q_f, \varepsilon) \quad q_f \in F$

Configuration finale de non acceptation : $\gamma_f(q_f, \varepsilon) \quad q_f \notin F$

Configuration Successeur après n transition

$$Q \times \Sigma^* \vdash_M^n \gamma$$

ssi $\gamma' = \gamma$ pour $n=0$

$$\exists \gamma'' \quad \gamma \vdash \gamma'' \text{ et } \gamma'' \vdash_M^{n-1} \gamma \quad \text{pour } n \neq 0$$

Notation : $\vdash_M^*$ Fermeture réflexive et transitive

$$L(M) = \{ w \in \Sigma^* \mid (q_0, w) \xrightarrow[M]{*} (q_f, \varepsilon), q_f \in F \}$$

Exemple : $M = (Q, \Sigma, \delta, q_0, F)$

$$\Sigma = \{a, b\}$$

$$Q = \{q_0, q_1\}$$

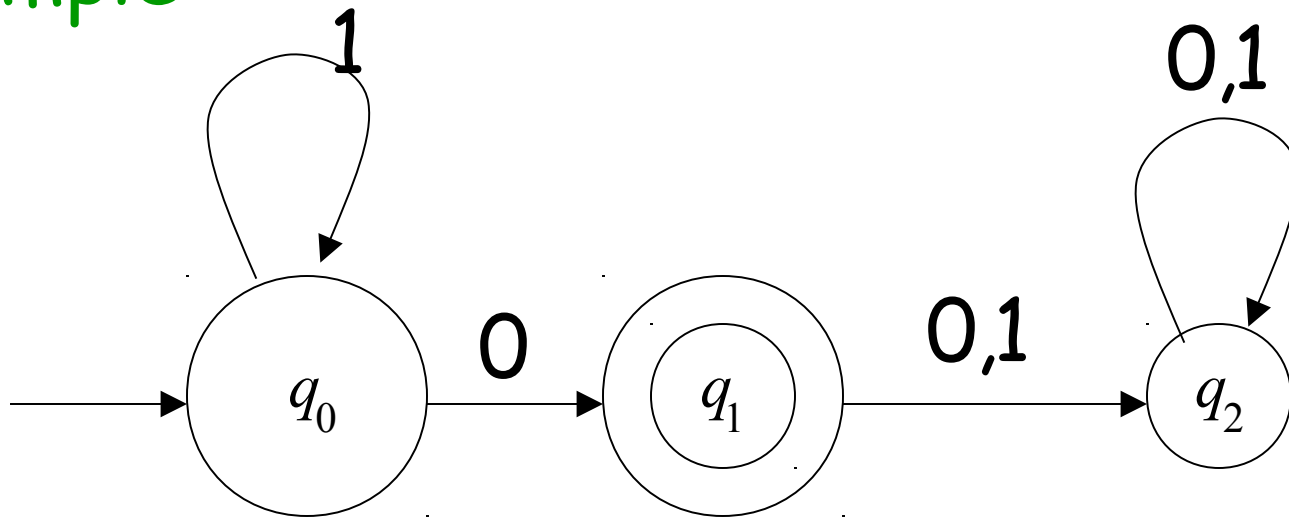
$$F = \{q_0\}$$

δ	a	b
q_0	q_0	q_1
q_1	q_1	q_0

- $(q_0, ababa)$
- (q_0, aba)

- Un automate fini déterministe est complet si δ est une fonction totale sur $Q \times \Sigma$
- De chaque état il part exactement une flèche étiquetée par chacune des lettres de l'alphabet Σ

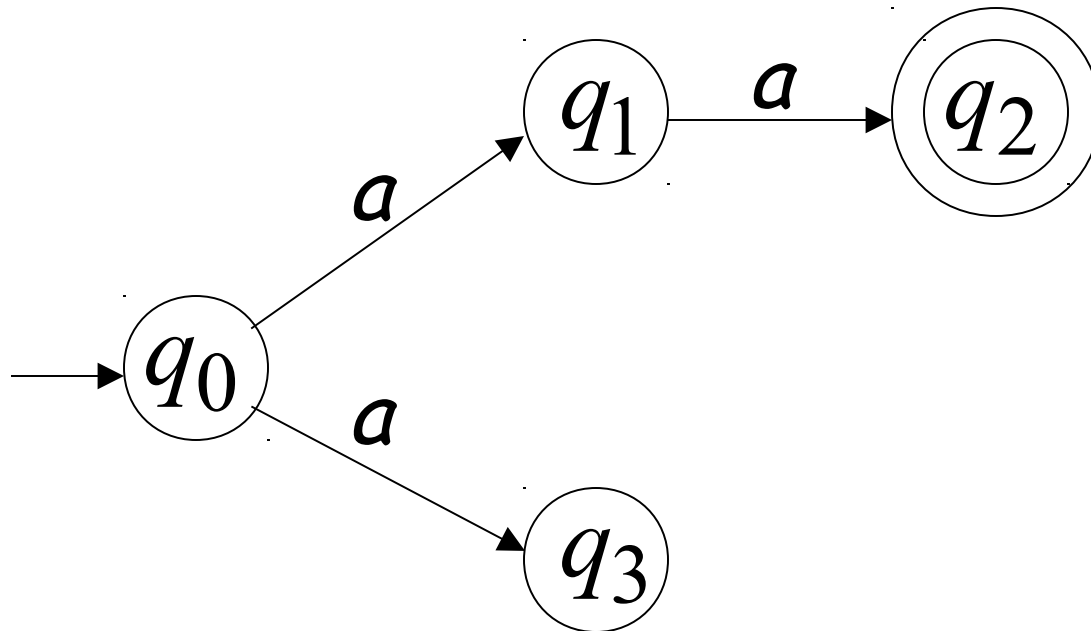
Exemple



- Ne pas confondre un automate M avec $L(M)$. En fait M est un programme et $L(M)$ est un ensemble de chaînes.
- L'état initial q_0 est de type état tan disque F est de type ensemble d'états (représente l'ensemble des états finaux)
- Est ce que a est un symbole ou une chaîne de longueur 1.
 - Ca dépend du contexte par exemple si a est utilisé comme entrée de la fonction δ ou δ^*

Automates finis non déterministe NFA

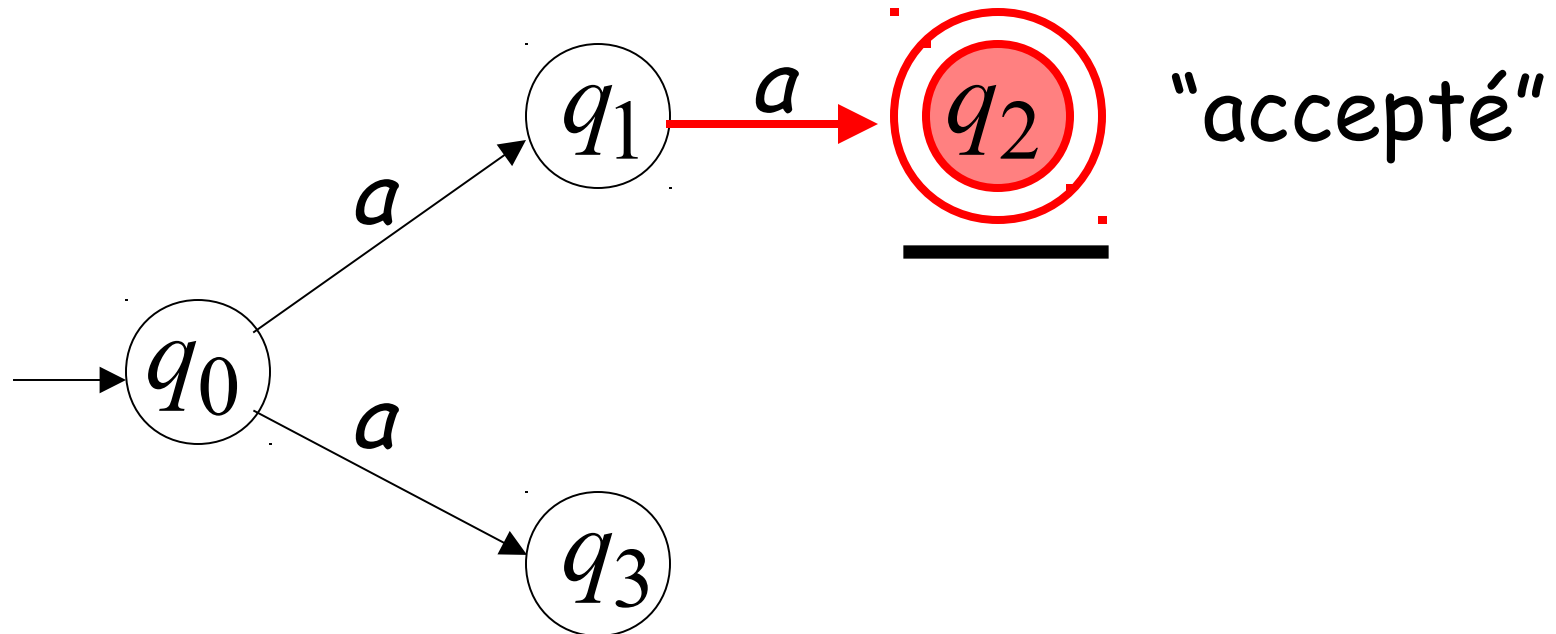
Alphabet = $\{a\}$



Automates
finis non déterministe
Premier choix

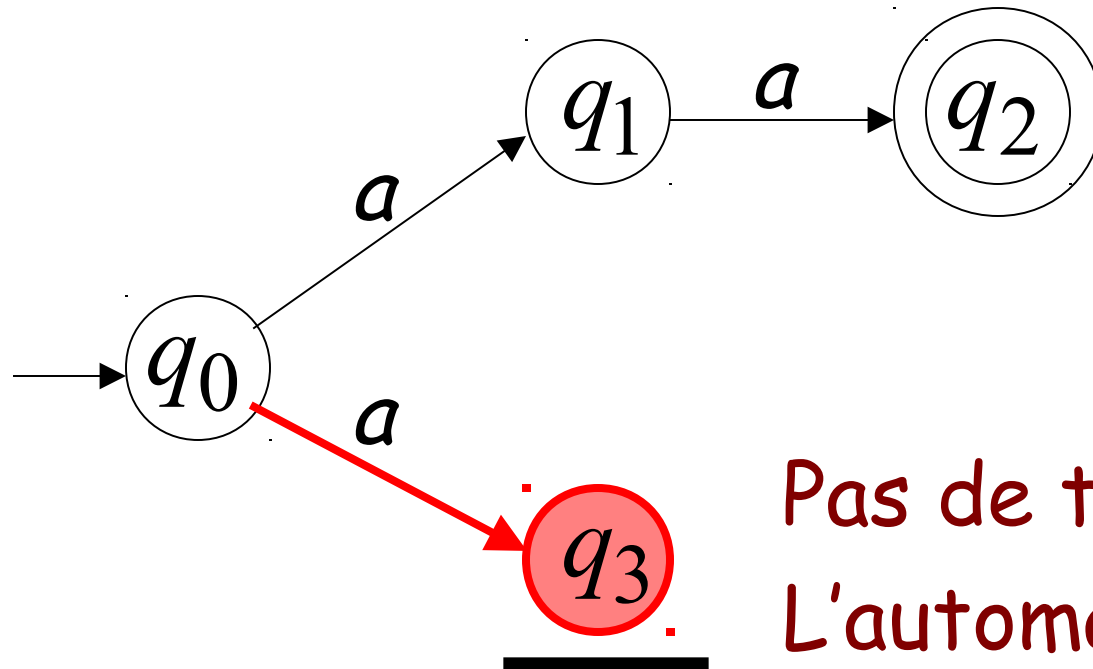


Tous les entrées sont consommées



Automates
finis non déterministe

Deuxième choix



Pas de transition:
L'automate se bloque
Rejet

un NFA accepte une chaîne:

Lorsqu'il existe un calcul dans le NFA
Qui accepte la chaîne

ET

Toutes les entrées sont consommées et
l'automate est dans un état d'acceptation

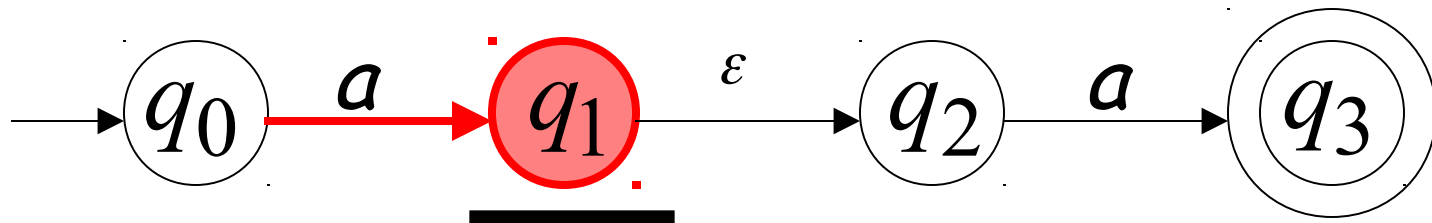
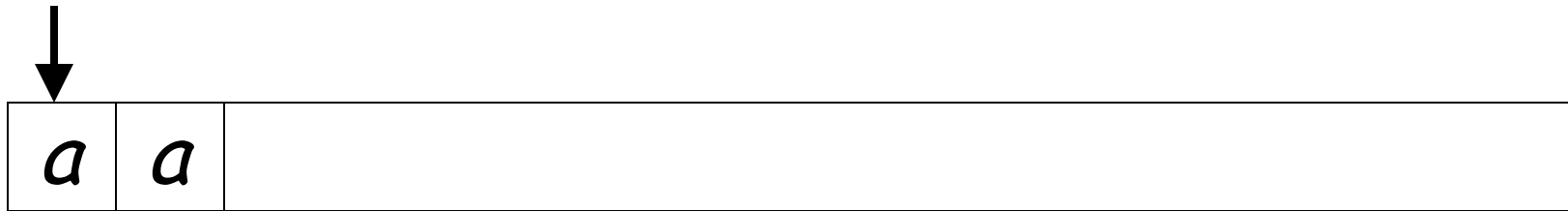
Un NFA rejète une chaîne:

Lorsqu'il n'ya aucun calcul dans l'NFA qui accepte la chaîne:

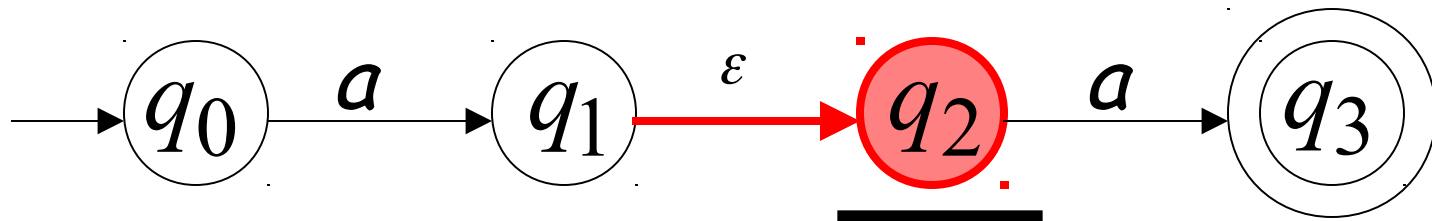
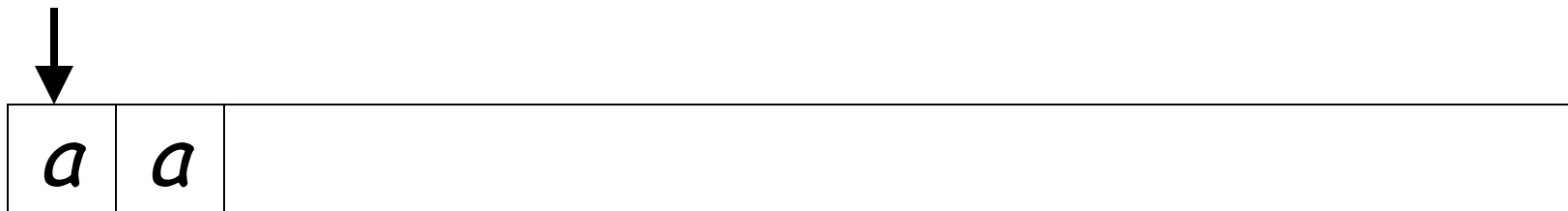
- Toutes les entrées sont consommées et l'automate n'est pas dans un état d'acceptation

OU

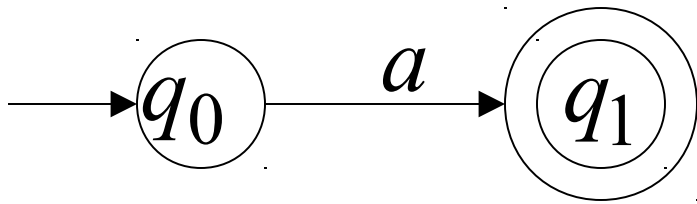
- L'entrée ne peut pas être consommée



(La lecture ne bouge pas)

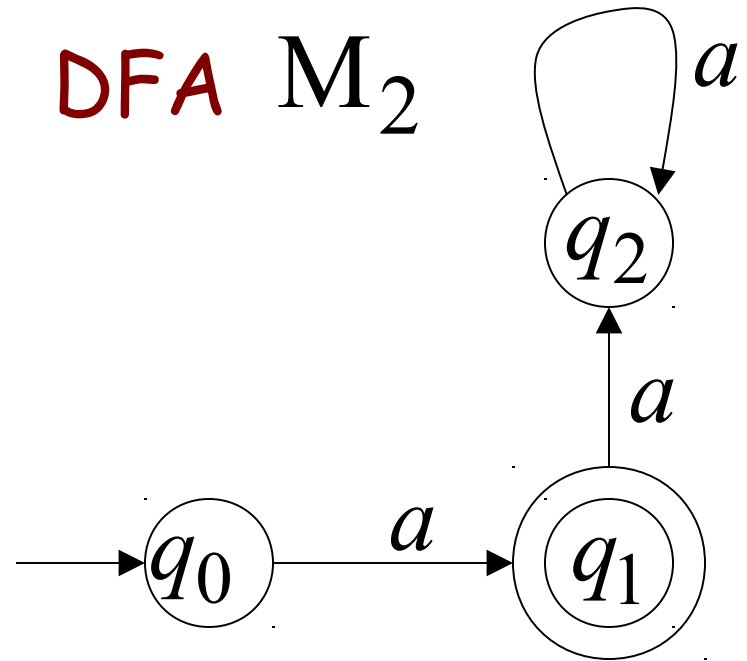


NFA M_1



$$L(M_1) = \{a\}$$

DFA M_2



$$L(M_2) = \{a\}$$

Définition formelle

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q : Ensembles d'états, $\{q_0, q_1, q_2\}$

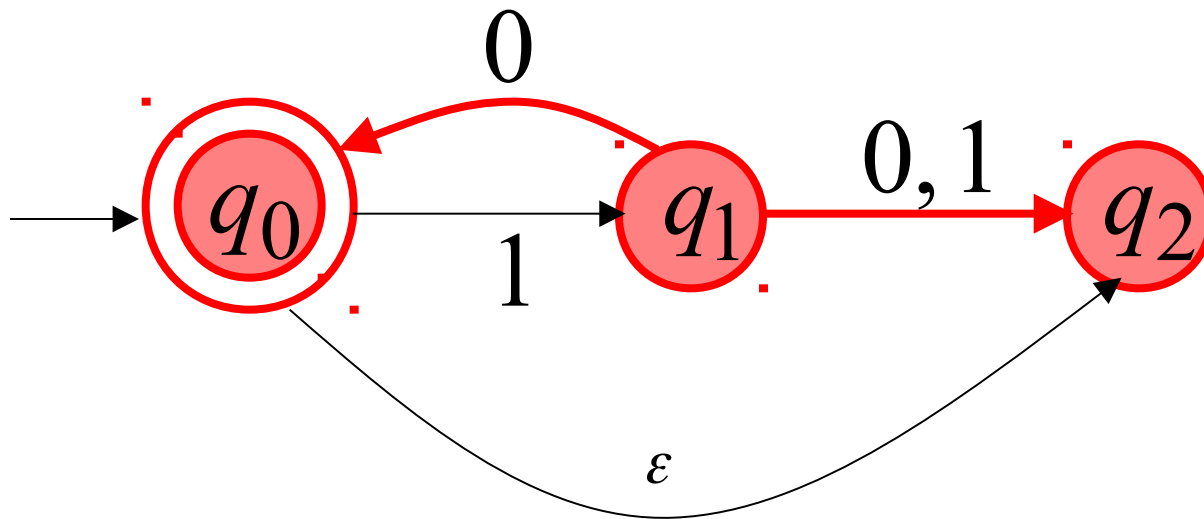
Σ : Alphabet d'entrées, $\{a, b\}$

δ : Relation de transition

q_0 : Etat initial

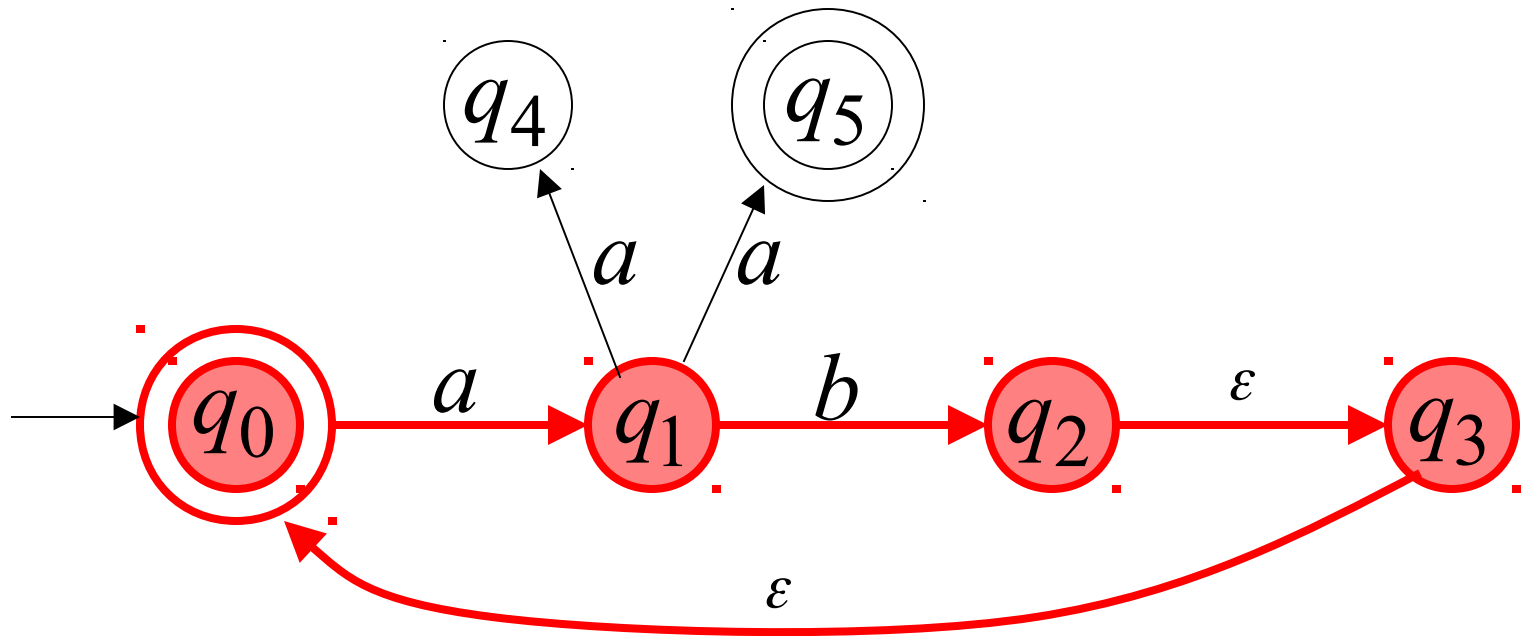
F : Ensembles d'états finaux

$$\delta(q_1, 0) = \{q_0, q_2\}$$



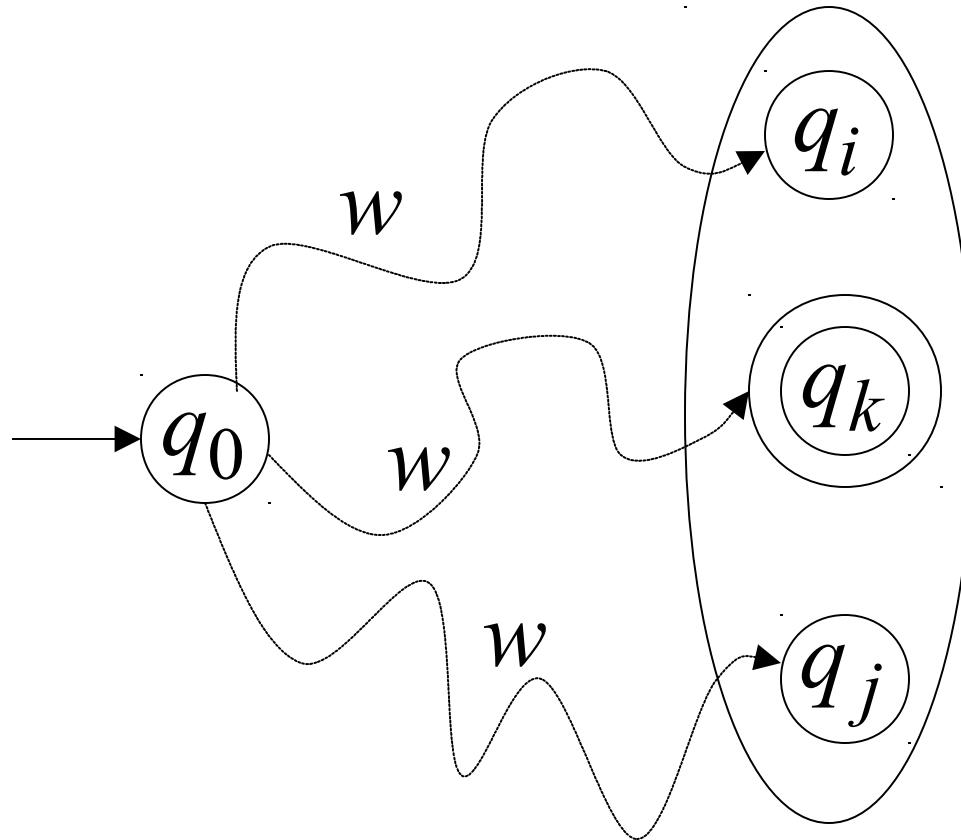
$$\delta(q_0, \varepsilon) = \{q_0, q_2\}$$

$$\delta^*(q_0, ab) = \{q_2, q_3, q_0\}$$



$$w \in L(M)$$

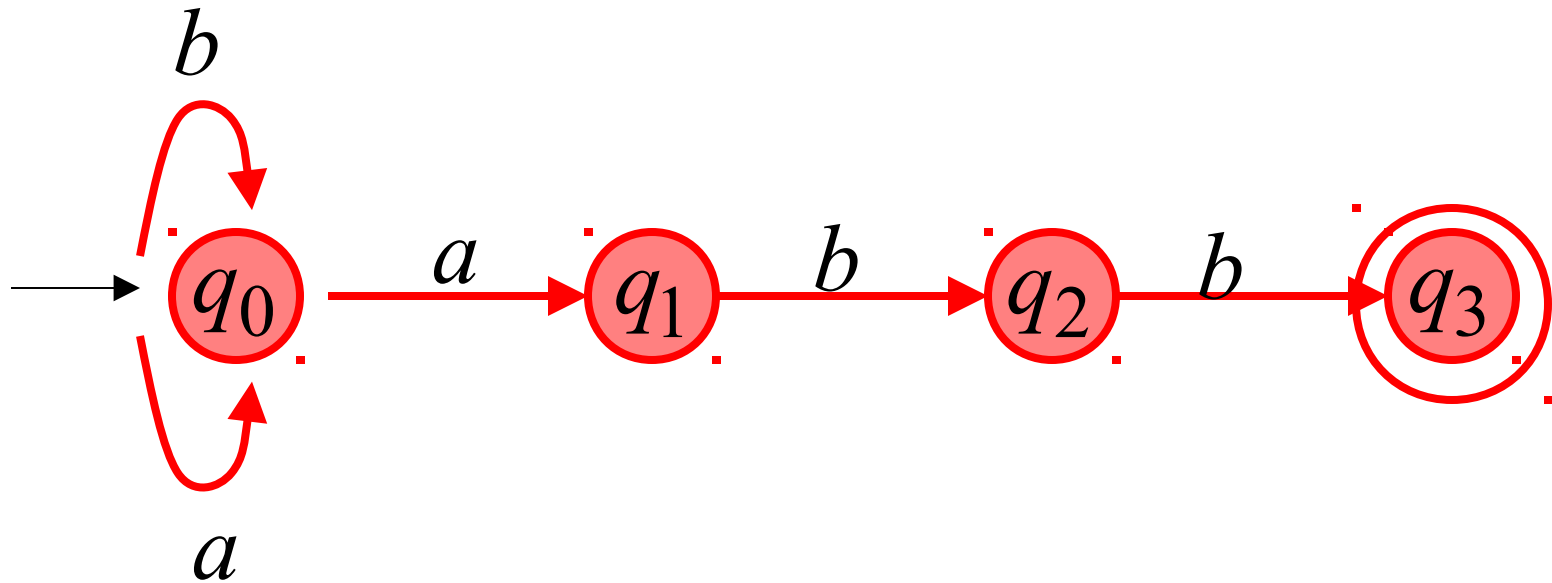
$$\delta^*(q_0, w)$$



$$q_k \in F$$

Exemple

Language($(a \cup b)^*abb$)



La machine M_1 est équivalente à la machine M_2 si

$$L(M_1) = L(M_2)$$

Théorème : Pour chaque automate fini non déterministe correspond un automate fini déterministe qui lui est équivalent.

Algorithme de transformation d'un NFA A en un DFA B

- $A = (Q, \Sigma, \delta, q_0, F)$
- $B = (Q', \Sigma, \delta', q'_0, F')$

$$E(q) = \{p \in Q \mid (q, \varepsilon) \xrightarrow[A]{*} (p, \varepsilon)\}$$

$Q' = 2^Q$ Puissance de l'ensemble (ensemble des parties de Q)

q'_0 $E(q_0)$: ε -fermeture de q_0

$$F' = \{q' : q' \cap F \neq \emptyset\}$$

Algorithme de transformation d'un NFA A en un DFA B

- $A = (Q, \Sigma, \delta, q_0, F)$
- $B = (Q', \Sigma, \delta', q'_0, F')$

$$E(q) = \{p \in Q \mid (q, \varepsilon) \xrightarrow[A]{*} (p, \varepsilon)\}$$

$Q' = 2^Q$ Puissance de l'ensemble (ensemble des parties de Q)

La construction de Q' est on-line un sous ensemble de 2^Q

q'_0 $E(q_0)$: ε -fermeture de q_0

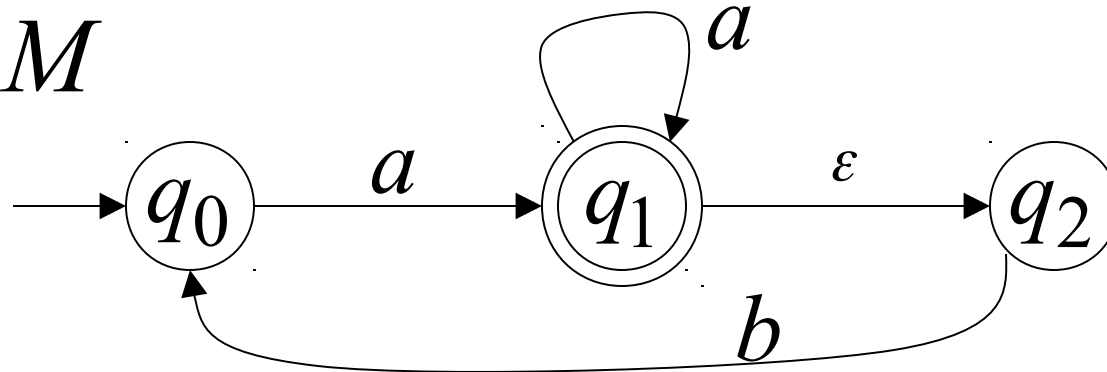
$$F' = \{K \subseteq Q : Q \cap F \neq \emptyset\}$$

Construction δ' pour tout $\sigma \in \Sigma$ faire :

$$\delta'(K, \sigma) = \bigcup \{ E(p) : p \in Q; \exists q \in K, \delta(q, \sigma) = p \}$$

$\delta'(K, \sigma) = \emptyset$ Possible

NFA M

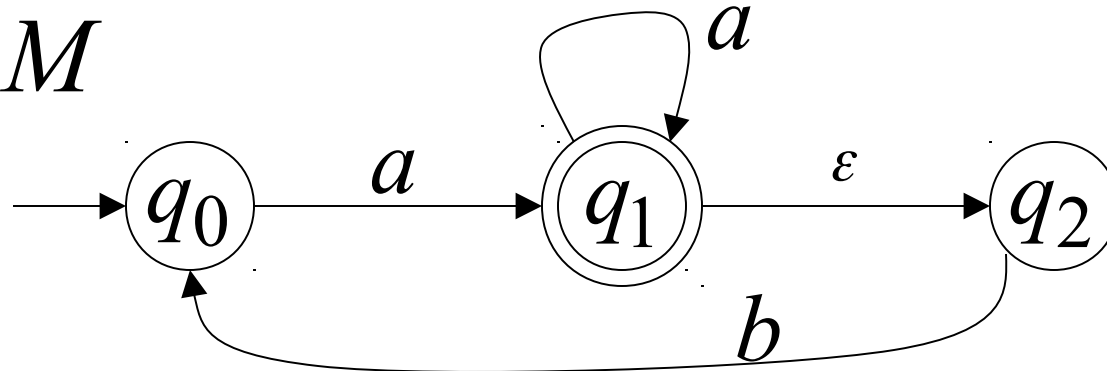


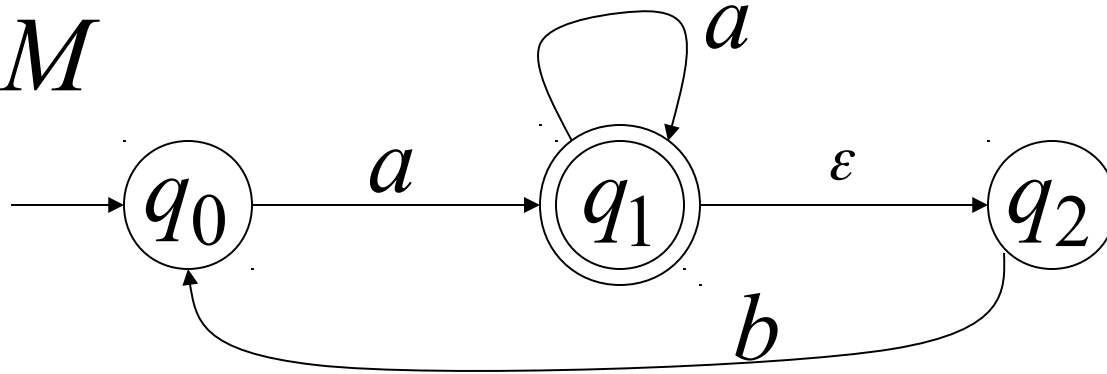
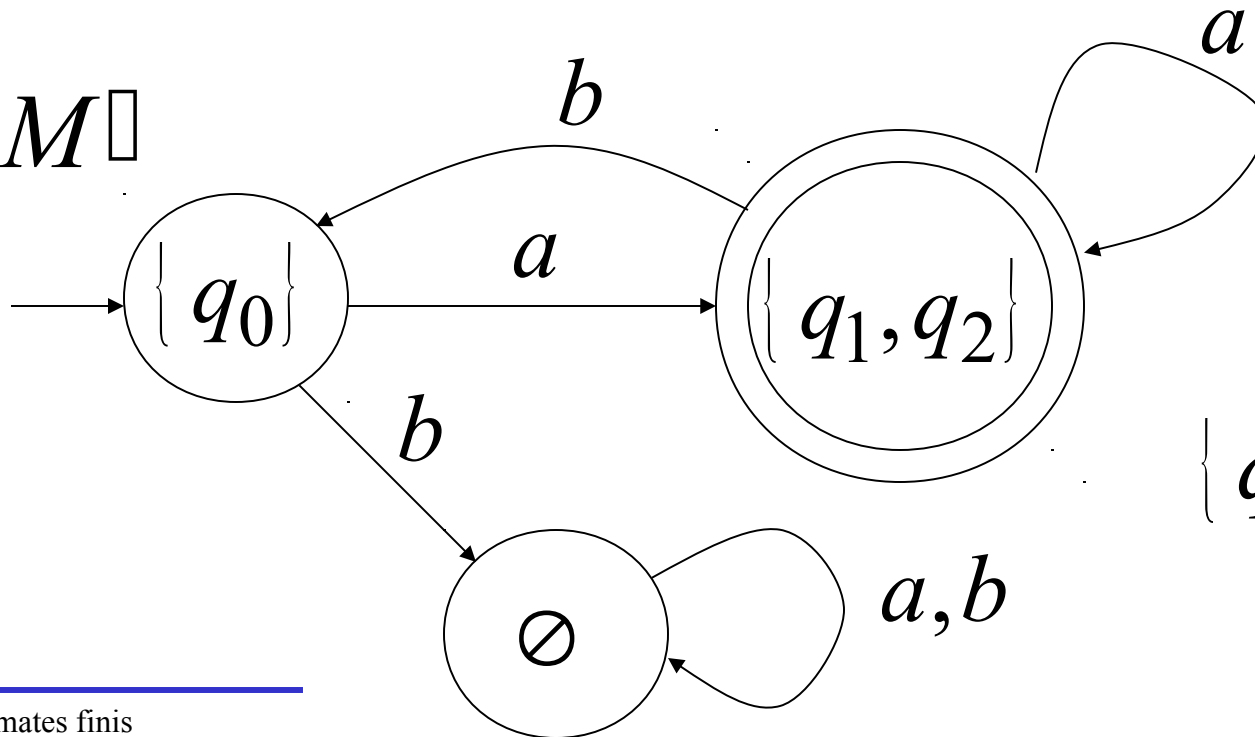
Construction δ' pour tout $\sigma \in \Sigma$ faire :

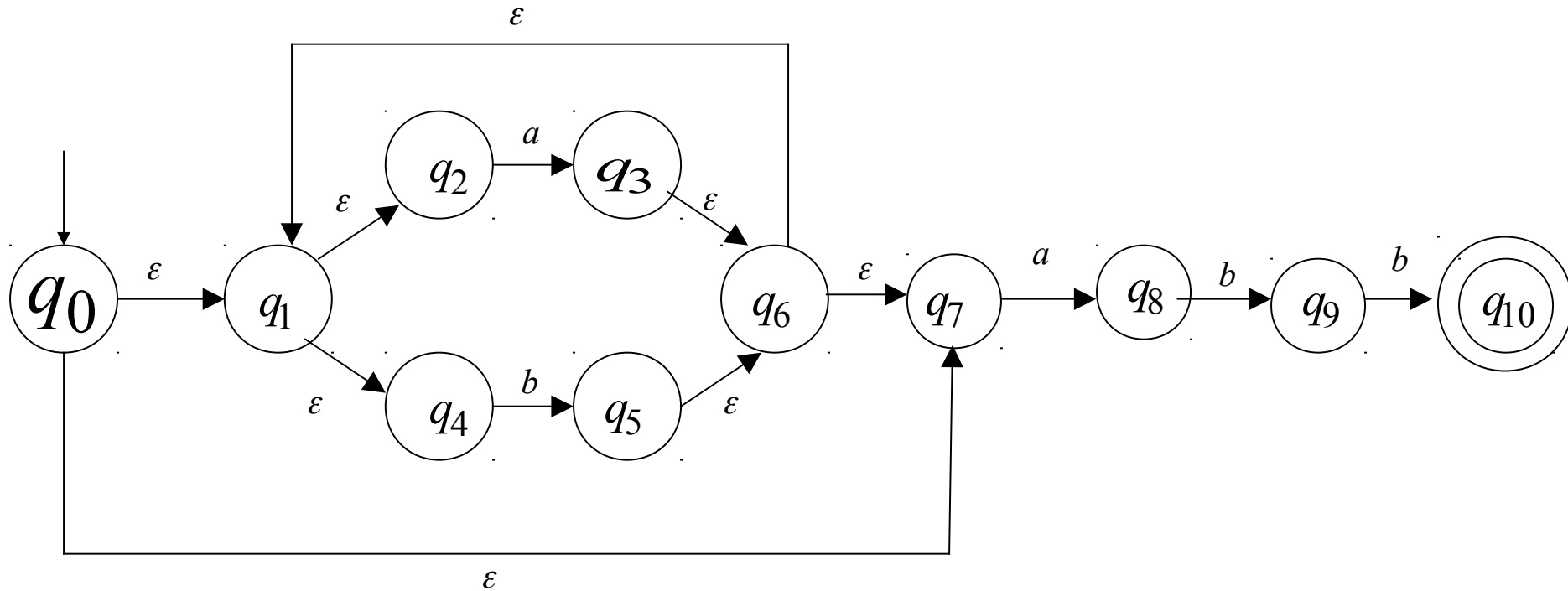
$$\delta'(K, \sigma) = \bigcup \{ E(p) : p \in Q; \exists q \in K, \delta(q, \sigma) = p \}$$

$\delta'(K, \sigma) = \emptyset$ Possible

NFA M



NFA M  $q_1 \in F$ **DFA** $M \square$  $\{q_1, q_2\} \in F \square$



Construire le DFA équivalent

- $A = \{q_0, q_1, q_2, q_4, q_7\}$
- $B = \{q_1, q_2, q_3, q_4, q_6, q_7, q_8\}$
- $C = \{q_1, q_2, q_4, q_5, q_6, q_7\}$
- $D = \{q_1, q_2, q_4, q_5, q_6, q_7, q_9\}$
- $E = \{q_1, q_2, q_4, q_5, q_6, q_7, q_{10}\}$

$$\delta(A, a) = E(q_3) \cup E(q_8)$$

$$\delta(A, a) = \{q_1, q_2, q_3, q_4, q_6, q_7\} \cup \{q_8\}$$

δ	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C

Propriétés des langages acceptés par un automate fini

La classe des langage acceptés par un automate fini est fermé par :

- Union;
- Concaténation;
- Kleen star;
- Complémentation;
- Intersection.

Dans chaque cas, on montrera comment construire
l'automate qui accepte le langage approprié

$$M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$$

L_1 langage accepté par M_1

$$M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$$

L_2 langage accepté par M_2

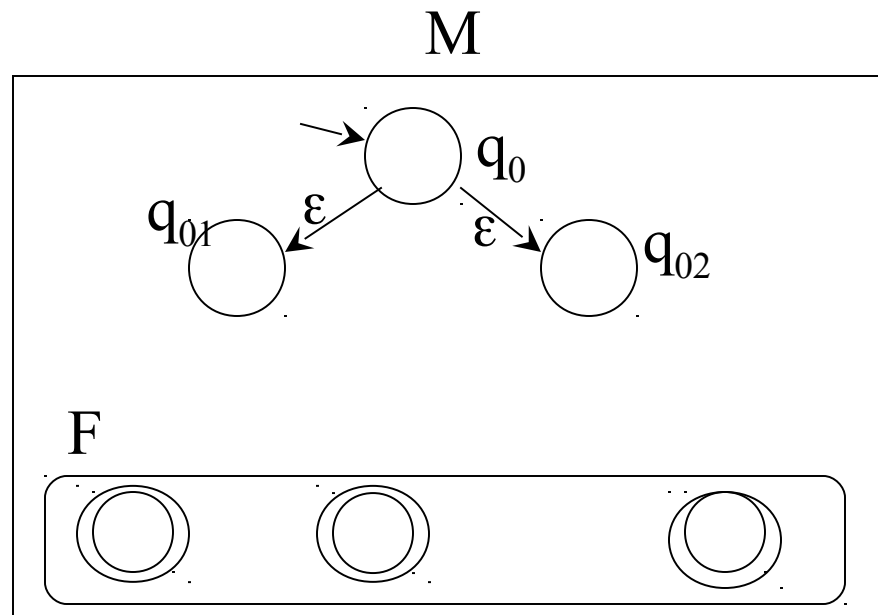
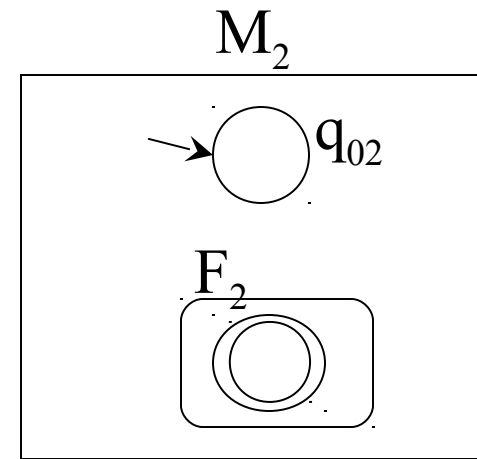
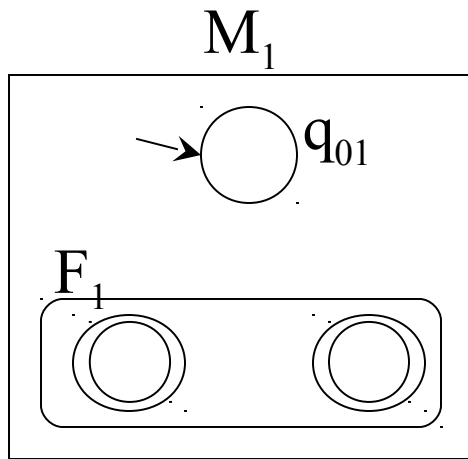
Soit M le NFA qui accepte $L(M_1) \cup L(M_2)$

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = Q_1 \cup Q_2 \cup \{q_0\}$$

$$F = F_1 \cup F_2$$

$$\delta = \delta_1 \cup \delta_2 \cup \{(q_0, \varepsilon, q_{01}), (q_0, \varepsilon, q_{02})\}$$



$$M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$$

L_1 langage accepté par M_1

$$M_2 = (Q_2, \Sigma, \delta_2, q_{02}, F_2)$$

L_2 langage accepté par M_2

Soit M le NFA qui accepte $L(M_1).L(M_2)$

$$M = (Q, \Sigma, \delta, q_{01}, F)$$

$$Q = Q_1 \cup Q_2$$

$$F = F_2$$

$$\delta = \delta_1 \cup \delta_2 \cup \{ F_1 \times \varepsilon \} \times \{ q_{02} \}$$

$$M_1 = (Q_1, \Sigma, \delta_1, q_{01}, F_1)$$

L_1 langage accepté par M_1

Soit M le NFA qui accepte $L(M_1)^*$

$$M = (Q, \Sigma, \delta, q_0, F)$$

$$Q = Q_1 \cup \{q_0\}$$

$$F = F_1 \cup \{q_0\}$$

$$\delta = \delta_1 \cup \{F \times \varepsilon\} \times q_0$$

$$M = (Q, \Sigma, \delta, q_0, F)$$

L langage accepté par M_1

Soit M' le NFA qui accepte $\Sigma^* - L(M_1)$

$$M' = (Q, \Sigma, \delta, q_0, Q - F)$$

Construction directe pour $L(M_1) \cap L(M_2)$?

Automate déterministe minimal

Définition (séparation)

Soit $(q_1, q_2) \in Q^2$: on dit que q_1, q_2 sont séparé par $w \in \Sigma^*$ si $\delta^*(q_1, w) \in F$ ou bien $\delta^*(q_2, w) \in F$ mais pas les deux en même temps.

Définition

q_1, q_2 sont inséparables si aucun mot de Σ^* ne les sépare
 $\delta^*(q_1, w) \in F$ ssi $\delta^*(q_2, w) \in F$

Définition (équivalence de nerode)

q_1 est équivalent à q_2 si q_1, q_2 sont inséparables (noté $q_1 \equiv q_2$)

Automate déterministe minimal

Définition (langage associé à un état)

Soit un DFA $A=(\Sigma,Q,\delta,q_0,F)$, on appelle langage associé à q et on note $L_q(A)$, le langage suivant :

$$L_q(A)=\{w \in \Sigma^* : \delta^*(q,w) \in F\}$$

$$L(A)=L_{q_0}(A)$$

Automate déterministe minimal

Problème :

Soit A un automate fini déterministe complet dont chaque état accessible depuis l'état initial. Construire un DFA minimal qui reconnaisse le même langage que A .

Idée : fusionner les états équivalents

Automate déterministe minimal

Définition (équivalence d'état) :

- Etant donné A un DFA, deux états p et q (noté $p \approx q$) sont équivalents si $L_p(A) = L_q(A)$.
- Ainsi $(p \approx q) \Leftrightarrow (\text{pour tout mot } w \in \Sigma^*)$

$$\begin{aligned} & \boxed{\delta^*(p, w) \in F \text{ et } \delta^*(q, w) \in F} \\ & \boxed{\text{ou}} \\ & \boxed{\delta^*(p, w) \notin F \text{ et } \delta^*(q, w) \notin F} \end{aligned}$$

Notation : si q est un état, on note [q] sa classe d'équivalence.
L'ensemble des états qui lui sont équivalents.

Automate déterministe minimal

Définition

Soit le DFA $A=(\Sigma,Q,\delta,q_0,F)$, l'automate minimal associé à A est

$$A_{\min}=(\Sigma,Q',\delta',[q_0],F')$$

- $Q'=\{[q] : q \in Q\}$
- $F'=\{[f] : f \in F\}$
- $\delta'=\{([p],a,[q]) \text{ tels que } \exists p' \in [p] \text{ et } \exists q' \in [q] \text{ et } (p',a,q') \in \delta\}$

Automate déterministe minimal

Propriété

- $A_{\min}$ reconnaît le même langage que A .
- Pour tout DFA B tel que $L(B)=L(A)$, le nombre d'états de B est supérieur ou égal à celui de A .
- $B_{\min}$ est le même que $A_{\min}$, On peut parler d'unicité à un renommage près.

Automate déterministe minimal

Algorithme de minimisation

- Itération $i = 0$ (construire deux classes d'équivalence : les états d'acceptation F et les états de non acceptation $Q-F$).

$$p \approx_0 q \text{ (} p \in F, q \in F \text{) ou (} p \notin F, q \notin F \text{)}$$

- Itération $i > 0$

$$p \approx_i q \text{ si :}$$

- $p \approx_{i-1} q$ Et
- pour tout $a \in \Sigma : \delta(p, a) \approx_{i-1} \delta(q, a)$

Arrêt de l'algorithme quand $\approx_i$ est identique à $\approx_{i-1}$

Supprimer tous les états morts et ceux non accessible depuis l'état de départ

Transformation d'un DFA en ER

Théorème :

Un langage est régulier si et seulement si il est accepté par un automate fini

Preuve :

if : Soit $M = (Q, \Sigma, \delta, q_0, F)$ un automate, on doit trouver un langage régulier R tel que $R = L(M)$.

On représente $L(M)$ comme l'union « fini »
de plusieurs langages simples

Transformation d'un DFA en ER

$$M = (Q, \Sigma, \delta, q_0, F)$$

Langage régulier R : $R = L(M)$.

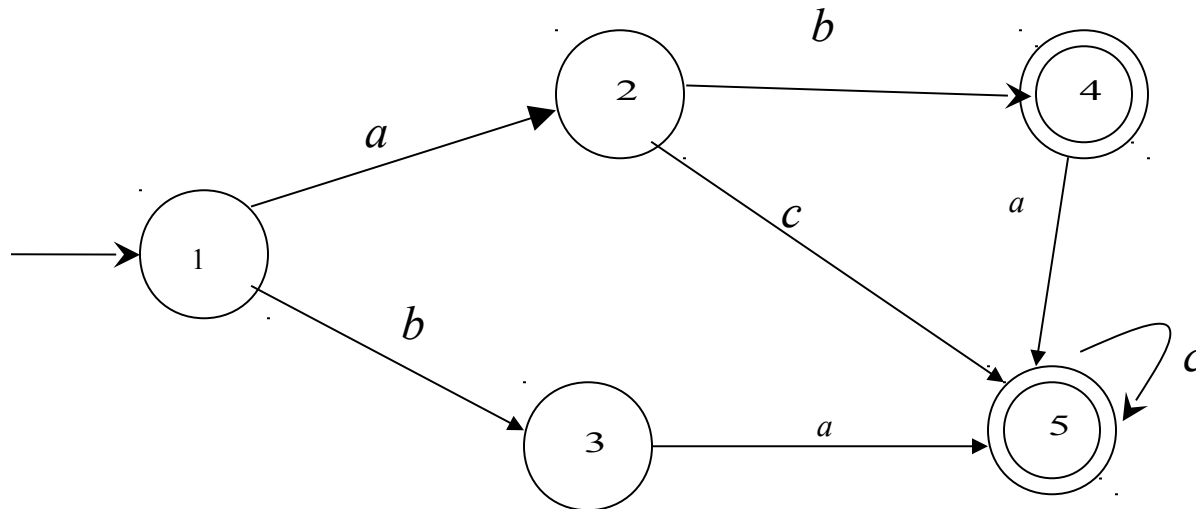
R : Union d'un nombre fini de simples langages.

Système d'équations associé à un automate :

$$\forall q_i \in Q, \quad L_i = \bigcup_{a \in \Sigma} aL_j \cup B_i \quad \delta(q_i, a) = q_j, B_i = \varepsilon \text{ si } q_i \in F.$$

Exemple

Construction d'une expression régulière pour le langage accepté par le DFA suivant.



$$\square L_1 = aL_2 + bL_3$$

$$\square L_2 = bL_4 + cL_5$$

$$\square L_3 = aL_5$$

$$\square L_4 = aL_5 + \varepsilon$$

$$\square L_5 = cL_5 + \varepsilon$$

Solution du système L_1 associé a l'état initial de M

Théorème:

Soit $(L_1, L_2, \dots, L_n)$ un système d'équations associé à un automate M . Ce système admet une solution unique et $L(M) = L_1$

Lemme d'arden

Soient $A, B \subseteq \Sigma^*$ tq $\varepsilon \notin B$ alors l'équation $L = A \cup BL$ admet une solution unique : $L = B^*A$

Et $L = A \cup LB$ admet une solution unique : $L = AB^*$.

Preuve:

B^*A solution,

$$L = A \cup BL, \quad B^*A = A \cup BB^*A = A \cup B^+A = (\varepsilon \cup B^+)A = B^*A$$

$B^*A \subseteq L$, inclus dans toute solution. $L = A \cup BL \Rightarrow A \subseteq L$,

$$BL \subseteq L$$

$BA \subseteq BL \subseteq L$, par récurrence, $B^iA \subseteq BL \subseteq L \quad \forall i$,

$$B^*A \subseteq L,$$

B A Solution unique:*

Lemme (Claim)

Si $K \subseteq BK$, $K, B \subseteq \Sigma^*$, $\varepsilon \notin B$ alors $K = \emptyset$.

Preuve

$$\begin{array}{l} K \subseteq BK \\ BK \subseteq B^2K \\ B^{n-1}K \subseteq B^nK \end{array} \Rightarrow K \subseteq B^n K \quad \forall n > 0$$

Soit $u \in K$ avec $|u|=p$, $u \in K$ alors $u \in B^{p+1}K$, donc $\exists w_1, w_2$, tq $u=w_1w_2$ avec $w_1 \in B^{p+1}$, $w_2 \in K$, donc $|w_1| \geq p+1$ car $\varepsilon \notin B$. Contradiction avec $|u|=p$.

Soit L une solution.

Posons $L = B^*A \cup K$, avec $B^*A \cap K = \emptyset$. $L = A \cup BL$

$$\begin{aligned} B^*A \cup K &= A \cup B(B^*A \cup K) \\ &= A \cup B^+A \cup BK \\ &= (\varepsilon \cup B^+)A \cup BK \end{aligned}$$

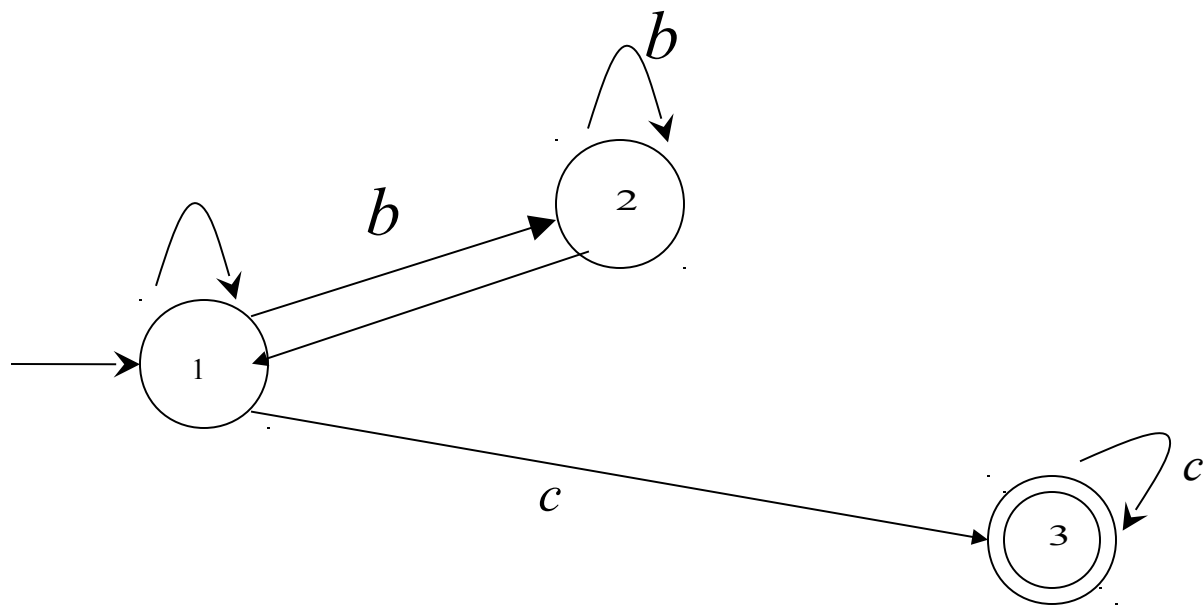
$$B^*A \cup K = B^*A \cup BK$$

$$K \subseteq BK$$

$$\varepsilon \notin B \quad \Rightarrow K = \emptyset,$$

Donc $L = B^*A$

Exemple



$$\begin{aligned} \square L_1 &= aL_1 + bL_2 + cL_3 \\ \square L_2 &= bL_2 + aL_1 \\ \square L_3 &= cL_3 + \varepsilon \end{aligned}$$

Exemple (suite)

D'après l'équation 3 on obtient:

$$L_3 = c^*$$

$$L_2 = b^* a L_1$$

$$\begin{aligned} L_1 &= a L_1 + b^* a L_1 + c c^* \\ &= b^* a L_1 + c^+ \end{aligned}$$

$$L_1 = (b^* a)^* c^+$$

Théorème de Pumping

On l'utilise pour prouver qu'un langage n'est pas régulier .

Soit L un langage régulier infini, pour chaque $w \in L$ avec $|w| \geq |Q|$
alors $\exists x, u, y \in \Sigma^*$ avec $u \neq \varepsilon$ tels que

$$w = xuy \quad (1)$$

$$|xu| \leq |Q| \quad (2)$$

$$|u| \geq 1 \quad (3)$$

$$xu^ny \in L \quad \forall n \geq 0. \quad (4)$$

Preuve

$w \in L, |w| \geq n, w = xuy$ tel que 1. $u \neq \varepsilon$ 2. $|xu| \leq n$ 3. $xuy \in L$

Soit le DFA ayant m états. Soit $|w| > m$. Considérons le chemin de l'état initial à l'état final. L'exécution de l'automate sur w doit passer par un même état au moins deux fois avec une partie non vide du mot séparant ces deux passages.

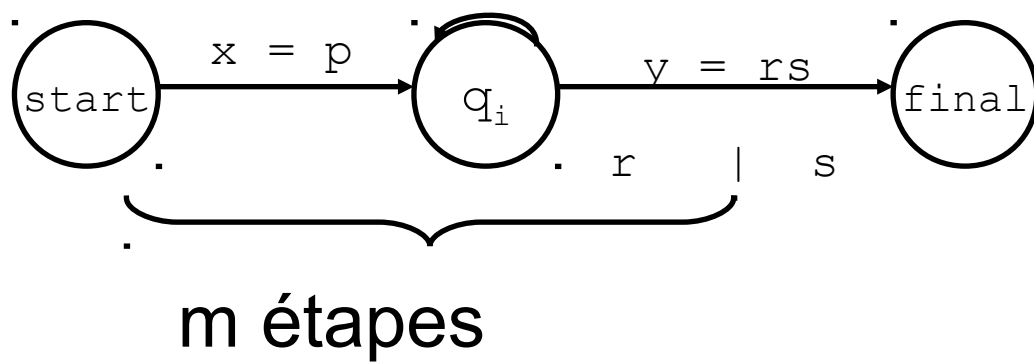
On peut écrire $w = opqr$, avec p correspond à une boucle, et $|opq| = m$ (le préfixe de taille m). Soit $x = o$, $u = p$, et $y = qr$.

1) Chaque boucle possède au moins un arc, $|p| > 0, u \neq \varepsilon$

2) $|xu| \leq m$ car $xu = op$ et $|opq| = m$

3) $xuy \in L$ car si p est une boucle elle commence de l'état s_i et $\underline{\delta}(s_i, p) = s_i$, et $\underline{\delta}(s_i, qr) = s_{\text{final}}$. $\underline{\delta}(s_{\text{start}}, x) = s_i$, et pour tout i $\underline{\delta}(s_i, u^i) = s_i$ c.q.f.d.

$$u = q$$



Preuve de non régularité

Pour prouver qu'un langage est non régulier on utilise le théorème de pumping comme suit:

On suppose que L est régulier (raisonnement par l'absurde),
soit $w \in L$ tel que $|w| \geq n$.

$w = xuy$, une décomposition de w en 3 sous chaînes, on suppose
que $u \neq \varepsilon$ et $|xu| \leq n$.

On doit démontrer que pour un certain i le mot xu^iy
n'appartient pas à L .

Exemple

Soit $L = \{a^n b^n \mid n \geq 0\}$, L est non régulier.

Il n'existe pas x, u, y tels que $xu^ky \in L \ \forall k$.

Supposons qu'il existe x, u, y .

3 cas sont possibles.

1^{er} cas. u dans a^n . On suppose que $u = a^p$, $x = a^r$ et $y = a^s b^t$ avec $p+r+s=t$
 $w = a^r a^p a^s b^t$, d'après le théorème, $a^r a^{np+s} b^t \in L \ \forall n$. contradiction $r+np+s \neq t$.

2^{ème} cas. u dans b^n symétrique.

3^{ème} cas. $u = a^p b^m$, $u^n = (a^p b^m)^n$. $xu^ny = a^{k-p} (a^p b^m)^n b^{k-m} \notin L$.

Il n'existe pas x, u, y avec $u \neq \varepsilon$, tel que $xu^ny \in L$ donc L n'est pas régulier.

De même on peut montrer que les langages suivants ne sont pas réguliers.

$L = \{w \in \{0,1\}^* \mid w \text{ contient le même nombre de 0s et de 1s}\}$

$L = \{w \in \{a,b,c\}^* \mid |w| = k^2\}$

$L = \{uu \mid u \in \{a,b\}^*\}$